

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ, СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**КЛИЕНТСКА БИБЛИОТЕКА ЗА ДОСТЪП ДО
БАЗИ ДАННИ ЧРЕЗ ПРЯКА РЕАЛИЗАЦИЯ НА
МРЕЖОВИТЕ ПРОТОКОЛИ НА POSTGRESQL
И MYSQL**

КУРСОВ ПРОЕКТ

по дисциплина „Надграждащи технологии за бази данни“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р инж. Надежда Кучмова

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА	2
II. ПРОЕКТИРАНЕ НА СИСТЕМАТА	3
III. ПРОГРАМНА РЕАЛИЗАЦИЯ	5
IV. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	8
V. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	10
5.1. Върхова потребена памет	13
5.2. Сравнение с доставчиковите библиотеки	13
5.3. Два дефекта, открити при това измерване	14
ЗАКЛЮЧЕНИЕ	16
ЛИТЕРАТУРА	18
ПРИЛОЖЕНИЕ	19

УВОД

Достъпът до релационна база данни от програма на C++ почти винаги минава през чужд слой: доставчикова клиентска библиотека, а често и диспечер на драйвери като ODBC. Всеки слой има собствен модел на паметта, собствена типова система и собствено копие на резултатните редове. Приложението плаща тази цена, без да я вижда.

Мрежовите протоколи на PostgreSQL и на MySQL са публично специфицирани [1, 2], тоест клиентът може да бъде написан срещу самата спецификация. Тогава всяко копие и всяко обръщане по мрежата е ред в собствения изходен текст и следователно може да бъде измерено.

Целта на проекта е да се проектира, реализира и измери клиентска библиотека на C++20, която говори двата протокола пряко. **Задачите** са пет: обосновка на избора между трите нива на достъп; проектиране на слоеста архитектура; програмна реализация за двата протокола; методика за измерване върху възпроизводима среда; измерване и анализ.

Обхватът е клиентската страна на комуникацията: установяване на връзка, удостоверяване, прости и подготвени заявления, декодиране на резултатни редове и управление на пул от връзки. Извън обхвата остават администрирането на сървъра, репликацията и разпределените транзакции.

Подготовка на автора по темата. Дипломната работа на автора за ОКС „бакалавър“ е ORM++, библиотека на C++ за обектно-релационно изображение със свързване при компилация. Авторът има приноси нагоре по веригата в MariaDB Connector/C. В производствена система е свел времето на едно заявление от 30 секунди до 1,5 секунди чрез преработка на самото заявление. Настоящият проект стои един слой по-надолу, при самия мрежов протокол.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА

ISO/IEC 9075 описва не само езика за заявки, но и начина, по който външна програма стига до данните: част 1 задава общата рамка [3], а част 3 определя *интерфейса на ниво извикване* (Call-Level Interface, CLI), през който приложение изпраща заявления, без те да са вградени в кода при компилация [4]. Съществено за проекта е това, което стандартът *не* нормира. Той нормира интерфейса, който приложението вижда, но не и протокола, по който клиентът и сървърът си говорят по мрежата. Протоколът е дело на всеки доставчик поотделно, затова преносимостта, която CLI обещава, се постига винаги чрез междинен слой към нестандартния протокол на конкретния сървър. ODBC е най-разпространената реализация на тази идея: приложението вика диспечер на драйвери, който зарежда драйвер за конкретния източник [5]. Върху този модел надграждат OLE DB, ADO и ADO.NET, а общото между тях е броят граници, през които минава един ред, невидим за приложението.

Задачата е една: заявление от програма на C++ и резултатни редове в типизиран вид. Решенията са на три нива. **А, слоест диспечер на драйвери:** една входна точка за много източници, но всеки ред минава през типа, договорен от стандарта, и през буфер, зададен предварително, а асинхронна работа няма [5]. **Б, доставчикова клиентска библиотека:** отпада един слой, но библиотеките са на C и налагат собствено управление на паметта на границата, а асинхронните им режими се различават твърде много, за да се обединят просто [6]. **В, пряка реализация на протокола:** отпадат всички междинни слоеве и декодирането работи върху собствения буфер без копие, но коректността става отговорност на проекта, включително редките части, които се откриват късно.

Проектът избира В. Основанието не е производителност сама по себе си, а наблюдаемост. При А и Б броят копия и обръщания е скрит зад чужд код и се оценява само косвено. При В всяко копие е ред в собствения изходен текст, тоест може да бъде премахнато или защитено с аргумент. Реализацията заема 5115 реда срещу публикуваните спецификации [1, 2]. Съпоставка на А и Б по брой слоеве и копия не е направена: тя изисква четене на чуждия изходен текст, а то не е правено, и празното място е по-полезно от предположение, което после ще бъде цитирано като измерване. Цената на избора е приета: две протоколни реализации и риск от разминаване с нови версии на сървърите.

II. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е разделена на пет слоя с еднопосочна зависимост. *Транспортът* притежава сокета и буферите и не знае нищо за съдържанието на байтовете. Над него стоят два *протоколни кодека*, по един за всеки сървър. *Декодерът на редове* превръща тялото на едно съобщение с данни в изгледи към стойности, без да ги копира. *Пулът* управлява живота на връзките, а всяка връзка носи собствен кеш на подготвените заявления. Най-отгоре *слоят за типово изображение* свързва стойностите от протокола с типовете на C++. Границата между два слоя е тесен договор, а не общ обект със състояние. Условието е всеки слой да се проверява самостоятелно (Табл. 1): от 91-те модулни теста нито един не изисква работещ сървър.

Таблица 1. Слоеве и начин на самостоятелна проверка

Слой	Как се проверява без сървър
Транспорт	двойка свързани сокета върху 127.0.0.1, отворена от самия тест
Протоколен кодек	байтови потоци, съставени поле по поле от спецификацията
Декодер на редове	изкуствени <code>field_view</code> в текстов и в двоичен формат
Пул и кеш	фиктивна връзка, която удовлетворява само изискването <code>poolable</code>
Типово изображение	списък от колони с имена, без протокол

Двата кодека нямат общ базов клас. Общото между тях е изразено като изискване към типа:

```
1 template <class C>
2 concept frame_codec = requires(byte_span in, const typename C::frame& f) {
3     typename C::frame;
4     { C::peek(in) } -> std::same_as<std::optional<typename C::frame>>;
5     { f.consumed } -> std::convertible_to<std::size_t>;
6     { C::describe(f) } -> std::convertible_to<const char*>;
7 };
```

Листинг 1: Договорът между двата кодека, изразен като изискване към типа

Изискването е проверимо, а не декоративно: в края на всеки от двата заглавни файла стои `static_assert(frame_codec<codec>)`, така че кодек, който престане да

удовлетворява договора, не се компилира. Ключовата дума е *reek*: функцията *разглежда*, но не консумира, тоест непълно съобщение струва празна стойност, никога загубен байт.

Всяка операция от свързването нагоре е съпрограма от типа `task<T>`. Типът е *мързелив*: създаването му не стартира нищо, а изчакването предава управлението чрез симетричен трансфер. Разликата не е стилистична: при верига от N изчаквания симетричният трансфер струва едно възобновяване, докато възобновяване отвътре на `await_suspend` натрупва N рамки в стека. Цикълът на събитията е един на нишка и не съдържа нито един ключ за заключване. Готовността на сокетите се проверява със `select`, единственото повикване с еднакво поведение на Windows и на POSIX; таванът от `FD_SETSIZE` дескриптора е записан като известно ограничение.

Декодерът не изгражда нови обекти, а издава `std::span<const std::byte>` и `std::string_view` към получаващия буфер; копие се прави само когато приложението изрично поиска притежание. Изгледите са валидни само докато трае извикването на обработчика на реда и това е част от публичния договор. Механизмът е в буфера: `recv_buffer` премества байтове само при уплътняване, а уплътняване се прави само при четене от сокета, тоест между две четения нищо не помръдва. Тестът `recv_buffer_keeps_views_stable_until_consumed` сравнява адреса на издадения изглед преди и след добавяне на байтове.

Заявка за връзка, която не може да бъде удовлетворена веднага, спира изпълнението на извикващата съпрограма, вместо да блокира нишка. Изчакването има срок, който се състезава със самото освобождаване: побеждава първото събитие, а губещото намира вече уредено състояние. Освободената връзка се предава *пряко* на най-дълго чакащия, а не се връща в списъка на свободните; защо това не е едно и също, е в Раздел III. Кешът е по връзка, защото подготвеното заявление и в двата протокола е свързано със сесията, а измественият запис се *връща* на извикващия, за да бъде затворен със `Close`.

Съответствието между колона и член на структура се задава като `constexpr` наредена n -торка от указатели към членове и се проверява при компилация: член, чийто тип няма декодер, не се компилира, а липсваща колона се съобщава по име веднъж за целия резултатен набор. Трите случая без точно съответствие между типовете са решени явно: числото с произволна точност се предава като десетичен текст, защото `double` губи точността, заради която типът съществува; празната стойност се чете само през `std::optional`; масивът се декодира само едномерен, а вложеният е грешка.

III. ПРОГРАМНА РЕАЛИЗАЦИЯ

Реализацията е на C++20 [7]: съпрограмите носят асинхронния модел, без да раздробяват логиката на протокола между обратни извиквания; `concept` изразява общото между двата кодека без виртуални функции; `std::span` и `std::string_view` са носителят на декодирането без копие. Сглобяването е с CMake и проектът няма нито една задължителна външна зависимост: на чиста машина стигат компилатор за C++20 и CMake. Това има цена, платена на три места. Тестовата рамка е собствена, 170 реда в `tests/check.hpp`, свързана с CTest. Измерването е със собствен часовник върху `std::chrono::steady_clock`. Хеш функциите, от които зависят двата обмена за удостоверяване, са в `src/crypto.cpp`: MD5, SHA-1 и SHA-256 [8], HMAC и PBKDF2, всяка проверена срещу публикуваните контролни вектори, а не срещу собствения си изход. Незадължителните зависимости се търсят с `find_package(... QUIET)` и никога не се свалят при сглобяване. OpenSSL е за TLS в самата библиотека [9]; `libpq` и `libmysqlclient` са само за измервателната програма. Ако някоя липсва, съответната част се пропуска с явно съобщение, а не се отгатва.

Непълно съобщение. Транспортът може да върне произволна част от пакет, така че кодекът трябва да може да спре и да продължи, без да губи състояние. Решението е кодекът да няма състояние изобщо: `peek_frame` гледа буфера и връща или цяло съобщение, или празна стойност, без да консумира нищо, а консумирането е отделна стъпка след обработката. Тестът `peek_frame_refuses_to_consume_a_partial_message` подава всяко възможно начало на едно съобщение, от нула до пълната дължина, и изисква празна стойност за всяко. Четенето на съобщение е изчакваем обект и `await_ready()` вика `peek_frame`, тоест в съпрограма се влиза веднъж на четене от сокета, а не веднъж на съобщение (Раздел 5.3).

Грешка от сървъра в средата на резултатен поток. Грешка, съобщена по правилата на протокола, задължава клиента да изчерпи обмена до синхронизиращата точка, преди връзката да бъде използвана отново. Реализацията записва грешката, но продължава да чете до `ReadyForQuery` и чак тогава хвърля изключение. Затова връзка, върху която е изпълнено грешно заявление, остава годна: интеграционният тест `pg_live_server_error_leaves_the_connection_usable` след грешка с код 42P01 изпълнява ново заявление по същата връзка.

Заемане на връзка от пула. Заемането се състезава с изтичане на срок. Съпрограмата не може да изчаква едновременно свободна връзка и таймер, затова едното от двете идва като обратно извикване, което урежда другото. Общото състояние се държи през `shared_ptr`, но извикването по срок държи `std::weak_ptr`, защото не се отменя при нормално даване на връзка и доживява до срока си (Раздел 5.3). Освобождаването предава връзката пряко на чакащия. Първата реализация я връщаше в списъка на свободните и събуждаше чакащия, което изглежда еквивалентно. Не е: съпрограмата, която освобождава връзката, продължава да се изпълнява преди събудения чакащ и си взема същата връзка обратно, така че единият работник завършва целия си дял, преди другият да получи изобщо една връзка. Поведението е записано като тест, а не като число: `pool_hands_a_released_connection_straight_to_the_waiter` пуска два работника по четири заемания при пул с размер едно и изисква ред на изпълнение `abababab`.

Обезсилване на кеша. Изместване от кеша връща измествения запис, върху който се изпраща `Close`; иначе всяко изместване би оставяло по едно заявление на сървъра. Ако сървърът отговори с код 26000, тоест не познава името на кешираното заявление, записът се изтрива и потокът се изпълнява повторно веднъж. Това се случва след `DISCARD ALL` или след смяна на сървъра зад пулър и извикващият не бива да го вижда.

Удостоверяване. За PostgreSQL са реализирани доверен вход, парола в чист вид, MD5 и SCRAM-SHA-256 [10]. Последният не е разширение, а необходимост: PostgreSQL 14 и следващите го използват по подразбиране, така че без него клиентът не може да се свърже със сървър, който не е нарочно отслабен. Обменът е проверен срещу публикувания в RFC 7677 пример: при фиксиран клиентски случаен низ реализацията възпроизвежда буква по буква и клиентското доказателство, и подписа на сървъра. За MySQL е реализиран `mysql_native_password`, включително отговорът на искане за смяна на приставката; `caching_sha2_password` не е реализирана и това се съобщава с ясна грешка, а не с мълчалив отказ.

TLS. И двата протокола сменят транспорта след кратък открит обмен, описан в самата спецификация. PostgreSQL приема `SSLRequest` вместо началното съобщение и отговаря с един байт `S` или `N`; при `S` следва ръкостискането на TLS, а началното съобщение и всичко след него са вече шифровани [1]. MySQL обявява `CLIENT_SSL` в ръкостискането; клиентът изпраща `SSLRequest` (същият пакет като отговора на ръкостискането, отрязан преди потребителското име) и чак тогава ръкостискането

на TLS, след което HandshakeResponse41 върви по шифрования канал [2]. Самият запис и четене минават през OpenSSL върху вече свързания сокет, с неблокиращо SSL_ERROR_WANT_READ и SSL_ERROR_WANT_WRITE към цикъла на събитията. Проверката на сертификата е по желание, защото контейнерите в `docker-compose.yml` използват самоподписан сертификат; без подаден `ca_file` и с `verify_peer` клиентът отказва този сертификат, което е проверено с модулен тест върху обратна връзка, а не срещу сървъра. Ако OpenSSL не е намерен при сглобяване, искането за TLS е ясна грешка, а тестовите, които го изискват, се пропускат.

Обработка на грешките. Разграничението е по типове. `protocol_error` означава байтов поток извън спецификацията и е неотстранима: позицията в потока е загубена. `io_error` е отказ на транспорта. `server_exception` носи полетата на грешката и е единствената, след която връзката се използва пак. Връзка с протоколна грешка или отказ на транспорта се отбелязва като негодна чрез флага `poisoned_`, а пулт проверява `is_usable()` при връщане и при заемане; тестът `pool_discards_a_connection_the_server_closed_underneath_it` убива връзката, докато тя е заета. За очаквано отсъствие се използва `std::optional`, не изключение.

IV. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

Проверката има две цели: коректност, тоест сървърът да връща същите стойности, които са били записани, и цена, тоест закъснителност, брой обръщания и цена на декодирането. Коректността е предусловие: измерване върху реализация, която дава различни резултати, няма смисъл.

Натоварването е пет случая, изпълнени от една програма върху една база: заявление за един ред през простия цикъл; същото през разширения при пропуск в кеша; същото при попадение; резултат от 5000 реда с четири колони, веднъж в текстов и веднъж в двоичен формат; заемане на връзка от пул при размер 1, 2, 4 и 8 и едновременност 1, 4 и 16. Тестовата таблица се създава от програмата в началото на всеки пуск.

Таблица 2. Конфигурация на машината и на средата при измерването

Компонент	Стойност
Машина и сглобяване	Windows 11 Pro N 10.0.26200, 12 логически ядра, g++ 15.2.0 (MinGW-w64), CMAKE_BUILD_TYPE=Release, C++20
Сървъри	postgres:16.4 (scram-sha-256) и mysql:8.0.39, в контейнери на същата машина, върху 127.0.0.1

Средата се вдига с Docker Compose, за да са фиксирани версиите и началното състояние на базите (Табл. 2). Че сървърите са на същата машина, а не на отделен хост, е съществено за тълкуването: част от разсейването идва от виртуализационния слой. Всеки от първите три случая се изпълнява 520 пъти, от които първите 20 се отхвърлят като загряващи; за големия резултат повторенията са 120 при 20 загряващи. За пула всеки работник изпълнява 40 заемания след предварително напълване, за да не влезе цената на свързването. Отчитат се медиана, 95-и процентил, средна стойност и стандартно отклонение: средна стойност без разсейване не позволява да се съди дали разлика е значима. Суровите проби се записват в `benchmarks/results/raw_samples.csv` при флаг `--raw` и от тях са изведени таблиците в Раздел V. Съобщенията се броят в клиента, не с часовник, което ги прави независими от машината.

Сравнение с `libpq` [6] и `libmysqlclient` тук не е направено: те не са инсталирани и програмата не отчита число. Следствието е, че на тази машина няма непроменена контрола, спрямо която да се мери собственият ѝ шум; съпоставката е в Раздел 5.2, върху контейнер.

[TODO: суровите проби в benchmarks/results/ не са прегенерирани: Docker не е наличен в тази среда]

V. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Коректността е предусловие за всичко останало и е проверена изцяло: 91 модулни случая без сървър, 7 интеграционни срещу PostgreSQL 16.4 и 7 срещу MySQL 8.0.39. Модулните случаи включват контролните вектори на MD5, SHA-1, SHA-256, HMAC-SHA-256, PBKDF2 и пълния обмен SCRAM-SHA-256 от RFC 7677 [10]; интеграционните проверяват, че записана стойност от тип `numeric`, `timestamp` и масив се прочита обратно със същата стойност през двоичния формат. Два от интеграционните случая са TLS върху същите сървъри, с `verify_peer` изключен, защото сертификатът е самоподписан; те се изпълняват само когато сървърът е достъпен и OpenSSL е свързан, иначе се пропускат с явно съобщение.

Таблиците в този раздел са от един пуск на приемащата машина, премерен наново след поправката на двата дефекта от Раздел 5.3.

Таблица 3. Едно логическо действие за един ред, микросекунди, $n = 500$ след 20 загриващи повторения. Съобщенията и обръщанията са преброени от механизма за проследяване

Вид заявление	Медиана	95-и пр.	Средно	Ст. откл.	Съобщ.	Обръщ.
PostgreSQL, прост цикъл	774	1019	803	105	5	1
PostgreSQL, разширен, пропуск	824	1066	854	114	12	1
PostgreSQL, разширен, попадение	742	959	766	98	7	1
MySQL, текстов протокол	863	1262	914	246	не е отчетено	не е отчетено

При трите случая за PostgreSQL стандартното отклонение е под четиринадесет на сто от средната стойност. При MySQL е близо 27 на сто и е най-голямото разсейване в таблицата. Правилото се прилага буквално: където разликата е под едно стандартно отклонение, извод не се прави. Първото такова място е самата Табл. 3. Трите вида заявление за PostgreSQL се различават с най-много 82 микросекунди при стандартни отклонения между 98 и 114, тоест измерването *не* подкрепя твърдение кой от трите е по-бърз. Причината е в средата: върху обратна връзка едно допълнително протоколно съобщение струва почти нищо.

Десните две колони дават две наблюдения. Първо, и трите случая струват едно обръщане по мрежата: разширеният цикъл *не* плаща допълнително обръщане при първото изпълнение, защото Parse, Describe, Bind, Execute и Sync се записват в един буфер и се изпращат заедно. Хипотезата за две обръщания би била вярна само за клиент, който изпраща Flush и чака отговор след Parse. Второ, кешът намалява съобщенията от 12 на 7, тоест с 42 на сто: спестени са Parse и Describe в изпратената посока и ParseComplete, ParameterDescription и RowDescription в получената. Тези пет съобщения не се превръщат в измеримо време тук, затова изводът е по-тесен от очаквания: дали спестените съобщения стават спестено време зависи от цената на едно съобщение по мрежата, а обратната връзка е най-неблагоприятният случай за кеша.

Таблица 4. Резултат от 5000 реда и четири колони, микросекунди, $n = 100$. Последната колона е времето само в декодиращите функции, измерено с отделен часовник

Случай	Медиана	95-и пр.	Средно	Ст. откл.	Декодиране
PostgreSQL, текстов	3812	4586	3888	524	1946
PostgreSQL, двоичен	3220	3802	3326	560	255
MySQL, текстов	6085	7985	6387	868	не е отделяно

Разликата в декодирането е 7,6 пъти: 1946 срещу 255 микросекунди за 5000 реда по три декодирани колони, тоест 130 срещу 17 наносекунди на стойност. Причината е в кода: двоичното декодиране на цяло число е едно зареждане и разместване на байтове, докато текстовото минава през разбор на десетичен запис, а двоичното декодиране на double е `std::bit_cast`, докато текстовото вика `strtod`. Общото време също е по-малко за двоичния формат, но печели *по-малко*, отколкото декодирането спестява: 592 микросекунди срещу 1691. Клиентът декодира ред, докато сървърът още произвежда следващите, тоест част от спестеното време освобождава време, което така или иначе щеше да се изчака.

Таблица 5. Закъснителност на заемане на връзка от пул, микросекунди

Размер	Едновременно	Медиана	95-и пр.	Ст. откл.	Чакали
1	1	0,9	1,8	0,4	0 от 40
1	4	2216	2524	278	159 от 160
1	16	11250	12534	1182	639 от 640
2	1	0,5	1,5	1,6	0 от 40
2	4	795	970	122	158 от 160
2	16	5884	6134	576	638 от 640
4	1	0,5	1,3	0,6	0 от 40
4	4	0,4	0,9	2,2	0 от 160
4	16	2737	3145	347	636 от 640
8	1	0,6	1,3	0,3	0 от 40
8	4	0,4	1,0	0,3	0 от 160
8	16	1012	1329	198	631 от 640

Когато едновременността не надхвърля размера на пула, заемането струва под една микросекунда по медиана и под две по 95-и процентил, а нито едно заемане не изчаква. Когато я надхвърли, заемането се превръща в чакане на опашка и цената му е времето за изпълнение на чуждо заявление, тоест хиляди микросекунди вместо части от една. Разликата между двата режима е повече от три порядъка и е най-голямата, която измерването откри. При едновременност 16 медианата пада от 11250 микросекунди при пул от една връзка до 1012 при пул от осем, тоест осемкратното увеличение намалява чакането 11,1 пъти, повече от пропорционално: при по-голям пул по-малка част от заявките изобщо стигат до опашката, което личи и от последната колона. Стандартното отклонение при насищане е около една пета от медианата или по-малко, тоест величината е чакане от самия пул, а не шум от средата.

Разликите се отнасят към три източника. **Брой копия:** двоичният формат печели 7,6 пъти в декодирането, разликата е изцяло в клиента и е точно това, което прякото говорене по протокол прави наблюдаемо, а именно един ред в Табл. 4 вместо скрито поведение в чужда библиотека. **Брой съобщения:** кешът спестява 42 на сто, точно по броене, но съответната разлика във времето е под разсейването и затова не се твърди. **Режим на работа:** заемането от пул има два режима с три порядъка разлика, а границата е едновременност, равна на размера на пула; това е единственият показател, при който изборът на параметър променя цената качествено.

Две неща не са измерени. Измерване при мрежа с ненулево закъснение не е правено,

а то е единственият начин да се провери дали спестените 42 на сто съобщения се превръщат в спестено време. Измерена е закъснителността на заемане от пул, но не и пропускливостта, тоест броят завършени заявления за единица време.

5.1. Върхова потребена памет

Измерена е на същата машина и в същото изпълнение като всички таблици дотук. Отчита се върхът на резидентното множество за целия ход на програмата, тоест покрива и резултата от 5000 реда, и случаите с пул, а не един отделен случай.

Върхът е **8,0 MiB** (8200 KiB). Големината се обяснява с решението резултатът да се предава на извикващия ред по ред през `field_view`, вместо да се натрупва целият в паметта: буферът расте с най-дългия ред, не с броя редове.

5.2. Сравнение с доставчиците библиотеки

Тези числа не са сравними с таблиците дотук и не бива да се четат заедно с тях. Машината, на която е измерено всичко останало, е Windows с MinGW и на нея нито `libpq`, нито `libmysqlclient` са инсталирани. Вместо да се слагат два клиентски стека на работна станция заради една съпоставка, тя се провежда в контейнер: Debian trixie, GCC 14.2.0, режим Release, а сървърите се достигат през мрежата на `docker compose`, не през публикуван порт. Затова медианите тук са около четири пъти по-малки от тези на приемащата машина. Съпоставими са *помежду си*: `conduit` и двете доставчикови библиотеки са измерени в един и същ процес, в едно и също изпълнение, срещу едни и същи сървъри.

Възпроизвежда се с една команда: `docker compose run --rm --build bench`.

Таблица 6. Един ред, същата заявка, същият сървър. Контейнер, GCC 14.2.0.

Клиент и път	Медиана	Ст. отклонение	<i>n</i>
conduit, прост протокол	211,6	52,3	500
libpq, PQexec	185,6	26,2	500
conduit, разширен поток с кеширан израз	186,1	37,2	500
conduit, текстов протокол MySQL	213,8	153,2	500
libmysqlclient, mysql_query	201,8	46,6	500

Стойностите са в микросекунди.

Четенето по редове на Табл. 6. PQexec използва простия протокол, затова честното сравнение е първите два реда: медианата на `libpq` е с 26,0 микросекунди по-ниска, тоест

14,0 на сто. Разликата обаче е под стандартното отклонение на `conduit`, което е 52,3, и по правилото, прилагано навсякъде в този документ, не носи твърдение. Третият ред не е същият път: разширеният поток с кеширан израз дава 186,1 срещу 185,6 на `PQexes`, тоест половин микросекунда при отклонения 37,2 и 26,2, а и сравнява два различни протоколни хода. При `MySQL` разликата е 12,0 микросекунди в полза на `libmysqlclient` при отклонения 153,2 и 46,6. И в двата случая разликата е далеч под разсейването и клиентите са неразличими.

Този пуск дава и контролата, която показва колко от една малка разлика е шум. Кодът на `libpq` не е пипан между предишния и този контейнерен пуск, а медианата му се измести от 194,5 микросекунди, записани в предишната редакция на този документ, на 185,6, тоест с 4,6 на сто. Разлика от този порядък между два реда в Табл. 6 е в рамките на това, което една и съща непроменена библиотека прави сама.

Върховата резидентна памет в контейнера е 11,3 MiB (11580 KiB) срещу 8,0 MiB (8200 KiB) на приемащата машина. Разликата е между две операционни системи и два компилатора, не между две реализации.

5.3. Два дефекта, открити при това измерване

Първи дефект: съпрограма на всяко протоколно съобщение. `read_message()` за `PostgreSQL` и `read_packet()` за `MySQL` бяха съпрограми, извиквани по веднъж на протоколно съобщение. Обичайният случай обаче е цялото съобщение вече да е в приемния буфер, защото едно четене от сокета носи много съобщения: резултат от 5000 реда пристига с далеч по-малко от 5000 четения. При това всяко съобщение създаваше рамка, а веригата от продължения растеше на дълбочина, вместо да се развива. Резултат от 5000 реда изчерпваше стека.

Сглобено с `GCC 12.2 (Debian bookworm)` измерването отказваше със `SIGSEGV`. С `GCC 14.2 (Debian trixie)` същият изходен текст минаваше. Първоначалният запис в този документ отдаваше това на разлика между двата компилатора и на разчитане на оптимизация. **Този запис беше грешен.** Грешка на компилатора е изключена чрез измерване: отделна минимална проверка на симетричния трансфер с 200 000 последователни изчаквания при `-O2` се изпълнява с `GCC 12.2`, без стекът да расте. Проверката е в хранилището, `benchmarks/symmetric_transfer_check.cpp`, и се повтаря с `сmake --build build --target symmetric_transfer_check`. Тоест самият

симетричен трансфер работи и дефектът е бил в съпрограмата на всяко съобщение, не в компилатора. Компилатор, който търпи един дефект, не е същото като липса на дефект.

Поправката. `read_message()` и `read_packet()` вече са собствени изчакваеми обекти, чието `await_ready()` прави надничането в буфера и връща истина, когато съобщението вече е там. Обичайният случай не спира изпълнението и не заделя нищо. Съпрограмата остава като `read_message_slow()` и `read_packet_slow()` и се влиза в нея веднъж на четене от сокета, а не веднъж на съобщение. И четирите места, от които `read_message()` се вика, са непроменени.

Втори дефект: обратно извикване по срок, което надживява чакация. След първата поправка програмата стигаше по-нататък и AddressSanitizer веднага откри втори, несвързан дефект. В `connection_pool::slot_waiter::await_suspend` обратното извикване за изтичане на срока при заемане държеше чакация през `shared_ptr`. Нищо не отменя това извикване, когато връзката бъде дадена по нормалния път, така че то доживява до срока си и може да се изпълни, след като чакащият вече е унищожен. Отчетът е четене на освободена памет, 40 байта, точно рамката на чакация.

Поправката. Обратното извикване вече държи `std::weak_ptr<waiter>` и го заключва с `lock()`. Това казва това, което е вярно: ако чакащият още чака, той е жив и достижим, а ако не е, извикването няма какво да прави.

Проверено състояние. GCC 12.2, режим Release: преди отказваше със SIGSEGV, сега завършва. GCC 14.2, режим Release: завършва. Приемачата машина, MinGW GCC 15.2, режим Release: завършва, а `ctest` минава 2 от 2. Под AddressSanitizer с GCC 12.2 програмата завършва и не се отчита нито една грешка, но за това трябва стек от 128 MB: собствената инструментация на санитайзера препълва подразбиращите се 8 MB. Това е свойство на санитайзера, а не остатъчен дефект, и се записва така, а не като чиста оценка без уговорка.

Колко се измести измерването. Медианата на простия цикъл за PostgreSQL е 774,2 микросекунди, а впуска преди поправките, записан в `benchmarks/results/run_log.txt`, беше 611,9. Причина не се твърди, защото не може да бъде подкрепена: на приемачата машина няма непроменена контрола, тъй като `librq` не е инсталирана там. Честното твърдение е, че числата се промениха след промяната, а мярка за собствения шум дава само контейнерният пуск. И трите дефекта дотук са намерени от измерване, а не от разсъждение, и нито един не беше уловен от тестовете.

ЗАКЛЮЧЕНИЕ

Проектът реализира клиентска библиотека на C++20, която говори пряко протоколите на PostgreSQL и MySQL, с асинхронен пул, кеш на подготвените заявления и декодиране без копие. Изборът е обоснован в Раздел I: прякото говорене прави цената на достъпа наблюдаема.

Предимства. Липсата на междинни слоеве премахва копия, които иначе са скрити, и измерването го потвърждава: декодирането от двоичен формат е 7,6 пъти по-евтино от текстовото, а изборът на формат е достъпен само защото клиентът сам изпраща Bind. Кешът спестява 42 на сто от протоколните съобщения за едно действие. Асинхронният модел не изисква нишка на връзка: всичко се изпълнява в един цикъл на събитията, без ключове за заключване, а проектът се сглобява само с компилатор и CMake.

Недостатъци и ограничения. Проектът поема две протоколни реализации и риска от разминаване при нови версии. Обхватът на протоколите не е пълен, съзнателно. Извън него остават: за PostgreSQL режимът COPY, LISTEN/NOTIFY, отмяната на изпълняващо се заявление по втора връзка и GSSAPI или SSPI; за MySQL двоичният протокол за подготвени заявления, компресията и caching_sha2_password. Декодирането обхваща едномерни масиви, а не вложени, и прехвърля отговорност за живота на данните върху приложението. TLS е реализиран през системния OpenSSL; без него библиотеката остава годна в доверена мрежа, а искането за TLS е ясна грешка. С проверка на сертификата е годна и през ненадеждна мрежа; без проверка е годна само срещу сървър, на който вече се вярва. Съпоставката с libpq и libmysqlclient е направена само в контейнер, защото на приемащата машина те не са налични, а разликите в нея са под разсейването, тоест твърдение за отношението към тях не се прави.

Инженерна трактовка. Разширеният цикъл *не* струва допълнително обръщане при първо изпълнение, защото петте съобщения се изпращат в един запис. Три дефекта дойдоха от измерване, а не от разсъждение, и нито един не беше уловен от тестовете: гладуване на единия работник при пул с размер едно; съпрограма на всяко протоколно съобщение, която растеше на дълбочина и изчерпваше стека при 5000 реда; и обратно извикване по срок в пула, което държеше чакащия силно и можеше да се изпълни, след като той е унищожен. И трите са поправени (Раздел 5.3).

По-нататъшна работа. Сравнение с доставчиците библиотеки на подходяща

машина; трети протокол, който би проверил доколко общият слой е общ; съвместимост с CLI [4].

ЛІТЕРАТУРА

- [1] PostgreSQL Global Development Group, “Frontend/Backend Protocol.” PostgreSQL Documentation, <https://www.postgresql.org/docs/current/protocol.html>.
- [2] Oracle Corporation, “MySQL Client/Server Protocol.” MySQL Internals Manual, https://dev.mysql.com/doc/dev/mysql-server/latest/PAGE_PROTOCOL.html.
- [3] International Organization for Standardization, “Information technology, Database languages, SQL, Part 1: Framework (SQL/Framework),” Tech. Rep. ISO/IEC 9075-1:2023, ISO/IEC, 2023.
- [4] International Organization for Standardization, “Information technology, Database languages, SQL, Part 3: Call-Level Interface (SQL/CLI),” Tech. Rep. ISO/IEC 9075-3:2023, ISO/IEC, 2023.
- [5] Microsoft Corporation, “ODBC Programmer’s Reference.” Microsoft Learn, <https://learn.microsoft.com/en-us/sql/odbc/reference/odbc-programmer-s-reference>.
- [6] PostgreSQL Global Development Group, “libpq: C Library.” PostgreSQL Documentation, <https://www.postgresql.org/docs/current/libpq.html>.
- [7] International Organization for Standardization, “Programming languages, C++,” Tech. Rep. ISO/IEC 14882:2020, ISO/IEC, 2020.
- [8] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” Tech. Rep. FIPS PUB 180-4, U.S. Department of Commerce, 2015.
- [9] OpenSSL Project, “OpenSSL Cryptography and SSL/TLS Toolkit.” <https://www.openssl.org/docs/man3.0/>.
- [10] T. Hansen, “SCRAM-SHA-256 and SCRAM-SHA-256-PLUS SASL Mechanisms,” Tech. Rep. RFC 7677, Internet Engineering Task Force, 2015.

ПРИЛОЖЕНИЕ

Изходният текст не се препечатва тук. Хранилището е публично: <https://github.com/Alex-Tsvetanov/conduit>. Табл. 7 дава указател към него; редовете са преброени с `wc -l`.

Таблица 7. Указател към изходния текст на проекта

Път	Какво съдържа	Редове
<code>bytes.hpp, task.hpp, net.*, tls.*</code>	буфери, <code>task<T></code> , сокет, TLS и цикъл на събитията	1509
<code>crypto.*, scram.*</code>	MD5, SHA-1, SHA-256, HMAC, PBKDF2, SCRAM-SHA-256	470
<code>codec.hpp, value.*, trace.*</code>	договорът между кодеците, декодерите, броене на съобщенията	612
<code>lru_cache.hpp, row_map.hpp, pool.hpp</code>	кеш с изместване, типово изображение, асинхронен пул	496
<code>pg/, src/pg_*.cpp</code>	кодек и връзка за PostgreSQL, двата цикъла, TLS	1176
<code>mysql/, src/mysql_*.cpp</code>	кодек и връзка за MySQL, текст протокол, TLS	852
<code>tests/</code>	91 модулни и 14 интеграционни случая, собствена рамка	2406
<code>benchmarks/bench.cpp</code>	измервателната програма, флаг <code>--raw</code>	613
<code>benchmarks/symmetric_transfer_demo.cpp</code>	проверка на асинхроничния трансфер расте по стек	85
<code>examples/demo.cpp</code>	показна програма с отпечатан протоколен обмен	286
<code>docker-compose.yml</code>	PostgreSQL 16.4 и MySQL 8.0.39, фиксирани версии, SSL	87

Схемата на измервателната таблица се създава от самата измервателна програма и е част от нейния изходен текст. Суровите проби от всяко повторение се записват при подаден флаг `--raw` във файла `raw_samples.csv`; пускът, от който са таблиците в Раздел V, произведе 5660 записа с полета `case`, `repetition` и `microseconds`.